A Practical Tutorial for Magic VLSI Layout

Using the GlobalFoundries 180 nm PDK (`gf180mcuD`)

James E. Stine
Oklahoma State University
Electrical and Computer Engineering Department
Stillwater, OK 74078 USA
`james.stine@okstate.edu`

May 2, 2026

Contents

1 Introduction **3**

2 Prerequisites and Setup **3**
2.1 Software . . . 3
2.2 Environment Check . . . 3
2.3 Create a Working Directory . . . 4

3 Launching Magic **4**

4 The Magic Interface **4**
4.1 Mouse Buttons . . . 4
4.2 Entering Commands . . . 5
4.3 The Box Tool . . . 5

5 Essential Command Reference **6**
5.1 File and View Commands . . . 6
5.2 Painting and Erasing . . . 6
5.3 Selection and Editing . . . 6

6 gf180mcuD Layer Reference **7**
6.1 Metal Stack Summary . . . 7

7 Labeling and Ports **8**

8 Step-by-Step: CMOS Inverter Layout in gf180mcuD **8**
8.1 Transistor Sizing . . . 8
8.2 Step 1 — Launch Magic and Create a New Cell . . . 9
8.3 Step 2 — Draw the nMOS Diffusion . . . 9
8.4 Step 3 — Draw the pMOS Diffusion . . . 9
8.5 Step 4 — Draw the Polysilicon Gate . . . 9

8.6 Step 5 — Add Diffusion Contacts . . . 10
8.7 Step 6 — Add a Poly Contact . . . 10
8.8 Step 7 — Add Substrate and Well Taps . . . 10
8.9 Step 8 — Route with Metal1 . . . 10
8.10 Step 9 — Add Labels . . . 11
8.11 Step 10 — Draw the N-Well . . . 11
8.12 Step 11 — Run DRC . . . 11
8.13 Step 12 — Save . . . 11

9 Design Rule Checking (DRC) 12

10 Extraction and Netlist Generation 12

11 Layout vs. Schematic (LVS) 13

12 Plotting and Printing Your Layout 13

13 Switch-Level Simulation with IRSIM 13
13.1 Extraction for IRSIM . . . 13
13.2 Running IRSIM . . . 14
13.3 IRSIM Command Files . . . 14

14 The Complete Design Flow 14

15 Tips and Common Pitfalls 15

16 Quick-Reference Cheat Sheet 16

Additional Resources 17

1 Introduction

Magic is an open-source VLSI layout editor originally written by John Ousterhout at UC Berkeley and actively maintained by Tim Edwards at http://opencircuitdesign.com. It is the workhorse of the open-source analog/digital IC design flow, providing interactive layout, built-in Design Rule Checking (DRC), and netlist extraction—all in a single tool. Magic receives frequent updates; always prefer version 8.3 or later.

This tutorial targets the **GlobalFoundries 180 nm MCU PDK** (`gf180mcuD`), an open-source process development kit available through the `open_pdks` installer and supported by Google's open-silicon initiative. The `gf180mcuD` process offers a 5-metal-layer CMOS flow (plus local interconnect) suitable for both digital and mixed-signal designs. All layer names, design rules, and command examples in this document refer to `gf180mcuD` unless otherwise noted.

Tip

Always prepare a **stick diagram** before opening Magic. Layout is a time sink; a hand sketch keeps you oriented and dramatically reduces trial-and-error. Think of it as your *game plan*: spending just 5 minutes with a stick diagram up front can save you hours of backtracking, erasing, and re-routing inside Magic. It forces you to think through transistor placement, wire routing, and layer choices *before* you are committed to geometry on screen—so that when you do open Magic, you know exactly what to draw and in what order. Keep your stick diagrams in a lab notebook or a digital file you can refer back to throughout the session.

2 Prerequisites and Setup

2.1 Software

You need Magic (8.3+), `open_pdks` with `gf180mcuD` installed, and optionally KLayout for printing or viewing GDS output. If you are using the IIC-OSIC-TOOLS Docker container (https://github.com/iic-jku/iic-osic-tools), all of this is pre-installed and pre-configured—skip ahead to Section 3.

For a native Linux build, follow the accompanying installation guide and confirm that the following environment variables are set (`setup.sh` handles this automatically):

```
export OPEN_CIRCUITDESIGN_ROOT="$HOME/ocd"
export PDK_ROOT="$OPEN_CIRCUITDESIGN_ROOT/share/pdk"
export PDK=gf180mcuD
```

2.2 Environment Check

Before starting any design work, verify that Magic can see the PDK:

```
ls $PDK_ROOT/gf180mcuD/libs.tech/magic/
# Should list: gf180mcuD.magicrc, gf180mcuD.tech, and related files
```

If the directory is empty or missing, re-run `open_pdks` for `gf180mcuD` or verify that `PDK_ROOT` is set correctly.

2.3 Create a Working Directory

Organize each design in its own directory to avoid file collisions:

```
mkdir -p ~/designs/inverter
cd ~/designs/inverter
```

3 Launching Magic

Always launch Magic with the `gf180mcuD` technology file so that the correct DRC rules, layer colors, and extraction rules are loaded:

```
magic -rcfile $PDK_ROOT/gf180mcuD/libs.tech/magic/gf180mcuD.magicrc
```

Magic opens two windows simultaneously:

- **Graphics (layout) window** — where you draw geometry.
- **Text (console) window** — where commands are typed and all feedback, errors, and DRC messages appear.

Important

Keep *both* windows visible at all times. DRC violations, extraction warnings, and all error messages appear *only* in the text window. If something unexpected happens in the layout, look there first. The graphical window will often appear completely normal even when serious problems exist — a DRC violation does not flash or highlight itself automatically, and an extraction warning produces no visible artifact in the layout view. The *only* indication that something has gone wrong is a message in the text window. Minimizing or hiding that window — even briefly — means you can miss critical feedback without realizing it. A few practical habits follow from this:

- **After every major operation** (placing a cell, running DRC, extracting a net), glance at the text window before continuing.
- **After a DRC run**, scroll through the full text output rather than stopping at the first clean result — later violations may be more serious.
- **When behavior seems wrong** — a command appears to do nothing, a shape doesn't appear where expected, or a tool exits immediately — the explanation is almost always already waiting in the text window.

Treat the text window not as a log to be consulted occasionally, but as a live diagnostic stream that runs in parallel with everything you do.

4 The Magic Interface

4.1 Mouse Buttons

The mouse is your primary layout instrument. Learn these three actions before anything else:

Button	Shorthand	Action
Left click	L	Sets the *lower-left* corner of the selection box
Right click	R	Sets the *upper-right* corner of the selection box
Middle click	M	Paints the layer the cursor hovers over in the palette, inside the current box

Tip

The middle-click paint trick is one of Magic's fastest workflows: draw your box with L and R, hover your cursor over the desired color in the layer palette, then middle-click to fill immediately.

4.2 Entering Commands

Commands can be entered in three equivalent ways:

1. **Single-character macros** typed in the graphics window (e.g., `g` toggles the grid, `v` zooms to fit).
2. **Long commands** prefixed with a colon typed in the *text* window (e.g., `:paint poly`, `:drc why`).
3. **Mouse buttons** directly in the layout window for painting and selection.

4.3 The Box Tool

All layout operations in Magic are relative to a rectangular *box*—the selection region shown as a white or yellow outline in the layout window. The default tool is the **BOX** tool, indicated by a crosshair cursor. To ensure you are in box mode:

```
:tool box
```

The `Space` bar cycles through all tools (Box, Wiring, Netlist, Pick). For basic drawing, always remain in Box mode.

Important

An accidental press of the `Space` bar will silently switch you out of Box mode—and because the cursor change is subtle, it is sometimes easy not to notice. If something stops working as expected—painting a layer produces no result, the box no longer resizes, or clicks seem to do the wrong thing—your first check should be whether you have inadvertently cycled to a different tool. This is an extremely common source of confusion, especially when typing commands while the cursor is over the layout window, since a space in what you *think* is the text window will instead be captured by the layout window and advance the tool cycle. To recover, either press `Space` repeatedly until the crosshair cursor returns, or type the explicit command:

```
:tool box
```

which will always return you to Box mode regardless of which tool is currently active. Make

it a habit to run this command any time behavior feels unexpected before attempting to diagnose anything else.

5 Essential Command Reference

5.1 File and View Commands

Command	Description
`:load` *name*	Load or create cell *name* (creates a new empty cell if none exists)
`:save` *name*	Save current cell to *name*`.mag`
`:quit`	Exit Magic
`:view` or `v`	Zoom to fit all painted geometry
`:f`	Zoom to fit the entire cell boundary
`:zoom` *n*	Zoom in by factor n (`:zoom 0.5` zooms out)
`z` / `Z`	Zoom window to box / zoom in $\times 2$
`:grid` or `g`	Toggle the visible grid
`:grid` *n*	Display a grid of $n\lambda \times n\lambda$
`:grid off`	Turn grid off
`:help [`*cmd*`]`	Print help for all or a specific command

5.2 Painting and Erasing

```
:paint <layer>      # paint <layer> inside the current box
:erase <layer>      # erase <layer> inside the current box
:erase *            # erase ALL layers inside the box
```

5.3 Selection and Editing

Command	Macro	Description
`:select`	`a`	Select geometry inside the box
`:copy`	`c`	Copy the selection
`:move`	—	Move the selection
`:delete`	`d`	Delete the selection
`:undo`	`u`	Undo last action
`:redo`	`U`	Redo
`:sideways`	—	Flip selection horizontally
`:upsidedown`	—	Flip selection vertically
`:clockwise`	—	Rotate selection 90° clockwise

Keys `Q W E R` (pressed in the graphics window) nudge the selection by 1λ left, up, right, and down respectively. The `.` (dot) macro repeats the last command—invaluable for repetitive placement tasks.

6 gf180mcuD Layer Reference

The table below lists the most commonly used layers for basic cell layout in `gf180mcuD`. The *Magic name* is exactly what you type after `:paint`; the description gives the physical meaning.

Magic Name	Type	Description
`nwell`	Implant	N-type well (body of pMOS transistors)
`ndiff`	Active	N-type diffusion (nMOS source/drain)
`pdiff`	Active	P-type diffusion (pMOS source/drain)
`poly`	Gate	Polysilicon gate and short-range routing
`ndc`	Contact	N-type diffusion-to-metal contact
`pdc`	Contact	P-type diffusion-to-metal contact
`nsc`	Contact	N-type substrate contact (tie to GND)
`psc`	Contact	P-type substrate/well contact (tie to VDD)
`pc`	Contact	Poly-to-metal1 contact
`li`	Metal	Local interconnect (TiN); use sparingly
`metal1`	Metal	Metal 1 — primary routing layer
`metal2`	Metal	Metal 2
`metal3`	Metal	Metal 3
`metal4`	Metal	Metal 4
`metal5`	Metal	Metal 5 (top layer)
`via1`	Via	metal1 → metal2
`via2`	Via	metal2 → metal3
`via3`	Via	metal3 → metal4
`via4`	Via	metal4 → metal5

Tip

Use metals for routing—not poly or local interconnect. Polysilicon has high resistance and capacitance. Local interconnect (e.g., `li`) is roughly 100× more resistive than `metal1` and should be reserved for short connections where metal is physically impossible. Route all signals on `metal1` and `metal2` whenever possible.

Beyond resistance, there is a strong hierarchical reason to be conservative with metals at the leaf-cell level: **stay at metal1 and metal2 in leaf cells and leave the upper layers free.** When you assemble leaf cells into blocks, and blocks into a full chip, the higher metal layers are what the block-level and chip-level routers depend on for power distribution, clock routing, and inter-block signals. If your leaf cells already consume metal3, metal4, or metal5, you create conflicts that are painful—sometimes impossible—to resolve without redesigning the cells from scratch. Think of the metal stack as a shared resource across the entire hierarchy: be a good citizen at the bottom, and the upper levels of the design will be far easier to close.

6.1 Metal Stack Summary

`gf180mcuD` provides five aluminum metal layers plus local interconnect between poly and metal1:

Layer	Magic Name	Typical Use
Local interconnect	`li`	Short hops; use sparingly
Metal 1	`metal1`	Leaf-cell routing (horizontal)
Metal 2	`metal2`	Leaf-cell routing (vertical)
Metal 3	`metal3`	Block-level routing
Metal 4	`metal4`	Power distribution
Metal 5	`metal5`	Top-level power/ground straps

For leaf cells (individual gates and standard cells), try to stay at metal1 and metal2 only. Reserving upper metal layers for higher levels of hierarchy makes block-level assembly and place-and-route much easier. Alternate metal1 (horizontal) and metal2 (vertical) where possible to minimize via counts and coupling.

7 Labeling and Ports

Labels attach logical names to layout geometry. They are required for power and ground nodes and for any signal you want to identify after netlist extraction.

```
:label <name>
```

Position the box over the geometry to label, then run the command. Follow these conventions:

- `vdd!` and `gnd!` — supply rails. The `!` suffix marks a node as *global*, meaning it spans the entire design hierarchy. Type these exactly as shown.
- Signal names must match the schematic exactly; LVS compares them character-for-character.
- Adopt a consistent capitalization style within a project and keep it.

Tip

Supply net naming and the global net convention. In Magic, appending `!` to a net name (e.g., `vdd!`, `gnd!`) is a convention that designates that net as *global*, meaning Magic will automatically connect every instance of that label across the entire hierarchy without requiring an explicit port or wire between cells. This can be convenient for small designs, but it is **optional** and entirely Magic-specific—it is not recognized by all tools, PDKs, or LVS configurations. Some flows define global supply nets through explicit port declarations, named power domains, or technology-file directives instead, and in those environments the `!` suffix may be ignored or behave unexpectedly.
The more portable and robust habit is to label your supply wires with consistent, simple names—`vdd` and `gnd`, or whatever your schematic uses—and ensure those names match exactly between the layout and the schematic netlist. Whether or not you use the `!` convention, the rules that always apply are:

- **Be consistent** — pick a naming convention and stick to it throughout the entire design.
- **Match the schematic** — LVS compares net names character-for-character; `VDD` and `vdd` are different nets.
- **Label explicitly** — a wire that is visually connected to a supply but carries no label will not be recognized as a supply net by the extractor.

8 Step-by-Step: CMOS Inverter Layout in gf180mcuD

This section walks through drawing a minimum-size CMOS inverter from scratch in `gf180mcuD`. Have your stick diagram ready before you begin.

8.1 Transistor Sizing

In `gf180mcuD` the minimum drawn gate length is $L_{\min} = 0.18\,\mu$m. A reasonable starting inverter uses:

Transistor	Type	Width	Length
nMOS	N-channel	$0.42\,\mu$m (minimum)	$0.18\,\mu$m
pMOS	P-channel	$0.84\,\mu$m ($2\times$ nMOS)	$0.18\,\mu$m

The pMOS is typically sized $2\times$ wider than the nMOS to equalize rise and fall times, since hole mobility is roughly half that of electrons.

8.2 Step 1 — Launch Magic and Create a New Cell

```
cd ~/designs/inverter
magic -rcfile $PDK_ROOT/gf180mcuD/libs.tech/magic/gf180mcuD.magicrc
```

In the text window, create a new cell named `inv`:

```
:load inv
```

8.3 Step 2 — Draw the nMOS Diffusion

Draw the nMOS active region in the p-substrate. This defines the source and drain of the nMOS transistor.

```
# nMOS source/drain region (in p-substrate)
:paint ndiff
```

8.4 Step 3 — Draw the pMOS Diffusion

Draw the pMOS active region above (or beside) the nMOS region. Leave enough space between them for the N-well enclosure and any substrate/well taps you plan to add.

```
# pMOS source/drain region
:paint pdiff
```

8.5 Step 4 — Draw the Polysilicon Gate

The poly must cross *both* active regions. It forms the transistor gate wherever it overlaps diffusion.

```
:paint poly
```

Draw one continuous vertical poly strip crossing both the `ndiff` and `pdiff` regions. The crossing automatically creates the gate of each transistor.

Tip

Keep dimensions at the DRC minimum where possible—smaller geometry means less parasitic capacitance and faster switching. That said, some layers are intentionally drawn wider than minimum: supply rails, for example, are made wider to reduce resistance and handle higher current without excessive voltage drop. When in doubt, start using minimum geometries and increase only when there is a specific reason to do so.

8.6 Step 5 — Add Diffusion Contacts

Connect the diffusion source/drain regions up to metal1 using diffusion contacts.

```
# nMOS drain and source contacts
:paint ndc

# pMOS drain and source contacts
:paint pdc
```

Place contacts inside the source and drain regions of each transistor, keeping them clear of the poly gate by the required spacing.

8.7 Step 6 — Add a Poly Contact

To route the gate (poly) to a metal wire for the input signal:

```
:paint pc
```

Place this at the top or bottom of the poly strip, wherever your input wire will land.

8.8 Step 7 — Add Substrate and Well Taps

Well and substrate taps are essential to prevent latch-up. The p-substrate must be tied to `gnd!` and the N-well to `vdd!`.

```
# N-type substrate contact (p-substrate tie to GND)
:paint nsc

# P-type well contact (N-well tie to VDD)
:paint psc
```

Place `nsc` in the p-substrate and `psc` where the N-well will be. These taps also help define the final footprint of the pMOS region.

8.9 Step 8 — Route with Metal1

Connect sources, drains, and gates using `metal1`:

```
:paint metal1
```

Make the following connections:

- nMOS source → `gnd!` (bottom power rail).

- pMOS source → `vdd!` (top power rail).
- nMOS drain + pMOS drain → output `Y`.
- nMOS gate + pMOS gate → input `A`.

8.10 Step 9 — Add Labels

```
# Position box over the VDD metal1 wire, then:
:label vdd!

# Position box over the GND metal1 wire, then:
:label gnd!

# Position box over the output metal1 wire, then:
:label Y

# Position box over the input wire (poly or metal), then:
:label A
```

Tip

When you need many similar labels—numbered pins, repeated bus signals, or sequentially named wires—typing each one individually in Magic can be tedious and error-prone. Because Magic's layout files (`.mag`) are plain text, a useful shortcut is to create one label, save the file, open it in any text editor, and use find-and-replace to duplicate or rename the label entries directly. When you reopen the file in Magic, all of the edited labels appear exactly as written—no retyping required. This is especially handy for buses where only a trailing index number changes (e.g., `data[0]` through `data[7]`).

8.11 Step 10 — Draw the N-Well

Now that the pMOS diffusion, well tap, and metal routing are all in place, you can see exactly how much area the pMOS region occupies. Draw the N-well last so you can size it precisely to enclose the `pdiff` and `psc` regions with the correct enclosure margin—no guesswork required.

```
:paint nwell
```

Extend the N-well until it fully covers the `pdiff` and `psc` geometry with sufficient overlap on all sides. DRC will immediately highlight any enclosure violation if the N-well is too tight.

8.12 Step 11 — Run DRC

```
:drc check
:drc why
```

Magic runs DRC continuously in the background; `:drc why` explains any violation under or near the current box. Fix all violations before proceeding to extraction.

Important

Do not proceed to extraction or LVS until DRC reports zero errors. A clean DRC is a hard prerequisite for a meaningful LVS comparison.

8.13 Step 12 — Save

```
:save inv
```

This writes `inv.mag` to your current working directory. Save frequently; Magic does not auto-save.

9 Design Rule Checking (DRC)

Magic performs DRC continuously as you draw, highlighting violations in real time. The following commands are most useful during layout:

Command	Description
`:drc check`	Force a complete DRC pass
`:drc why`	Explain the DRC error inside the current box
`:drc find`	Move the box to the next DRC error
`:drc count`	Report the total number of DRC errors

Common DRC errors in `gf180mcuD` and how to fix them:

- **Poly too close to diffusion edge** — increase the spacing between the poly gate end and the edge of the diffusion region.
- **Contact enclosure violation** — the surrounding diffusion or metal region is too narrow; enlarge the enclosing layer around the contact.
- **N-well enclosure of pdiff insufficient** — the N-well must extend beyond the p-type diffusion by the required enclosure distance; widen the N-well.
- **Metal spacing violation** — two metal1 wires are too close together; increase the gap between them.
- **Via overhang violation** — vias in `gf180mcuD` require a specific enclosure overhang on both the lower and upper metal layers; ensure the metal extends far enough beyond the via on all four sides.

10 Extraction and Netlist Generation

Once DRC is clean, extract a netlist from the layout for LVS and post-layout simulation.

```
:extract all
:ext2spice hierarchy on
:ext2spice format ngspice
:ext2spice
```

This produces `inv.ext` (the intermediate extraction database) and `inv.spice` (a SPICE netlist with parasitic capacitances). You can also use the course convenience script if it is available in your environment:

```
ext4mag inv.mag
```

This script generates both `.gds` and `.sim` files and optionally a SPICE deck (answer `N` when prompted about SPICE if you do not need it yet).

11 Layout vs. Schematic (LVS)

LVS verifies that your layout correctly implements the intended schematic. It is one of the most critical steps in the design flow. Many professional organizations list skipping LVS as a termination offense. Run LVS after every cell passes DRC, and use it as a progressive debugging tool as you assemble larger designs cell by cell.

Run LVS with Netgen:

```
netgen -batch lvs "inv.spice inv" \
    "$PDK_ROOT/gf180mcuD/libs.tech/netgen/gf180mcuD_setup.tcl"
```

A passing result prints **Circuits match** with zero discrepancies. If it fails, Netgen lists the mismatched devices or nets—trace these back to errors in your layout or labels.

Tip

Most companies draw the schematic first, then create the layout to match it. A SPICE netlist from xschem or a textual HDL description both serve as valid references for Netgen.

12 Plotting and Printing Your Layout

Always use KLayout to produce layout documentation—never screenshots. KLayout renders GDS layers accurately and exports to postscript or PDF.

```
# Inside Magic: write GDS
:gds write inv

# In a terminal: open with KLayout
klayout inv.gds
```

To convert KLayout's postscript output to PDF:

```
ps2pdf ./inv.ps inv.pdf
```

KLayout should be installed as part of your EDA tool suite. If layer colors do not appear correctly, import the `gf180mcuD` technology file via **Tools** → **Manage Technologies** → **Import Technology**.

13 Switch-Level Simulation with IRSIM

After a successful LVS, validate digital logic function with IRSIM, a switch-level simulator that models transistors as ideal switches. IRSIM is much faster than full SPICE simulation and is well-suited for verifying logic correctness from extracted layout.

13.1 Extraction for IRSIM

```
# In Magic
:extract all
:ext2sim
```

This produces `inv.sim`, which IRSIM reads directly.

13.2 Running IRSIM

```
irsim $PDK_ROOT/gf180mcuD/libs.tech/irsim/gf180mcuD.prm inv.sim
```

13.3 IRSIM Command Files

Automate stimulus with a `.cmd` file to build a reusable test vector set (often called a *golden file*). A simple inverter command file:

```
# inv.cmd
stepsize 100
| Initial state: A = 0, expect Y = 1
set A 0
c
print Y
| Toggle: A = 1, expect Y = 0
set A 1
c
print Y
| Toggle back: A = 0, expect Y = 1
set A 0
c
print Y
```

Load and run the command file inside IRSIM:

```
source inv.cmd
```

A correct inverter outputs `Y` high when `A` is low, and `Y` low when `A` is high.

14 The Complete Design Flow

The full open-source IC design flow for `gf180mcuD` is:

1. **Draw schematic** in xschem or write a SPICE text netlist.
2. **Simulate schematic** with ngspice (SPICE) or IRSIM for initial functional verification.

3. **Draw stick diagram** to plan transistor placement and wire routing.
4. **Create layout** in Magic using the `gf180mcuD` technology.
5. **Run DRC** — fix all violations before moving forward.
6. **Run LVS** with Netgen against the schematic netlist.
7. **Extract and simulate layout** — run ngspice or IRSIM on the extracted netlist to capture parasitics and verify timing and power.
8. **Plot** the layout using KLayout for documentation and submission.

Important

Never skip LVS. This is not optional. LVS is your guarantee that what you drew is actually what you designed. Skipping it is one of the most costly mistakes you can make in IC design—a layout that passes DRC but fails LVS will produce a chip that does not work, and at fabrication costs of thousands to hundreds of thousands of dollars per run, there is no opportunity to just "try again." The consequences in industry are severe: I have heard from colleagues that several companies explicitly list bypassing LVS as grounds for immediate dismissal or a reprimand. Run LVS after every cell passes DRC—not just at the top level, not just at the end of the project, but cell by cell as you build up the design. Catching a net mismatch in a single gate takes minutes to fix; finding the same error buried in a full-chip LVS failure can take days, weeks, or months.

Be warned: **getting LVS to pass is genuinely extremely hard.** A failing LVS report is not a simple error message—it is a puzzle. The tool gives you clues in the form of mismatched nets, unmatched devices, and node-count discrepancies, and it is up to you to decode what they mean and trace them back to the root cause in your layout or labels. Approach it like a debugging session: read the output carefully, form a hypothesis, make one change, and re-run.

Commercial tools such as Mentor/Siemens **Calibre DRV** are the industry gold standard for this, offering rich GUI integrations that highlight mismatches directly in the layout and schematic side by side, making the puzzle significantly easier to solve. Open-source engines such as Netgen are capable and free, but they are typically slower and produce less guided output than their commercial counterparts. That is not a reason to avoid them—it is a reason to stay vigilant and optimistic. Read the output carefully, work through it methodically, and remember that a passing LVS is always achievable if your layout is correct.

15 Tips and Common Pitfalls

- **Always use a stick diagram.** The single most effective way to avoid wasting hours in Magic is to know exactly what you intend to draw before you open the tool.
- **Route on metal, not poly.** Poly has significantly higher resistance and capacitance than metal. Use it only for the transistor gate and very short local connections.
- **Stay at metal1/metal2 in leaf cells.** Reserve upper metal layers for block-level routing and power. Wires are the dominant contributor to cell area; keeping them at the lowest layers preserves routing resources for higher levels of hierarchy.

- **Alternate metal directions.** Use metal1 for horizontal routing and metal2 for vertical routing (HVH style) to minimize via counts and reduce coupling.
- **Use pitch as a design-wide parameter.** Establish a consistent wire pitch for all wires in a cell and stick to it. Uniform pitch makes cell abutment and later place-and-route much smoother.
- **Label everything.** Supply rails (`vdd!`, `gnd!`) and all I/O signals must be labeled for extraction and LVS to work correctly.
- **Save frequently.** Use `:save` after any significant change. Magic does not auto-save.
- **Use scripts.** Extraction, LVS, and GDS export are all scriptable. A well-written script saves hours across a semester of design work. You can find the location of any installed script with `which` *`scriptname`*.
- **Post problems publicly.** Include a screenshot and a step-by-step description of what you tried. Other students learn from your problems, and public posts build a searchable archive of solutions for the entire class.

16 Quick-Reference Cheat Sheet

Keyboard Macros

Key	Long form	Action
`v`	`:view`	Zoom to fit all geometry
`g`	`:grid`	Toggle grid
`a`	`:select`	Select inside box
`c`	`:copy`	Copy selection
`d`	`:delete`	Delete selection
`u`	`:undo`	Undo
`U`	`:redo`	Redo
`z`	—	Zoom window to box
`Z`	`:zoom 2`	Zoom in $\times 2$
`.`	—	Repeat last command
`Q/W/E/R`	—	Nudge selection 1λ left/up/right/down
Space	`:tool box`	Cycle tools (return to Box)

Key Commands

To launch Magic, use (where `$PDK_ROOT` is set by `setup.sh`):

```
magic -rcfile $PDK_ROOT/gf180mcuD/libs.tech/magic/gf180mcuD.magicrc
```

Alternatively, you can omit the `-rcfile` flag by placing a `.magicrc` file in your home or working directory that loads the technology automatically. Add the following line to `~/.magicrc`:

```
tech load $env(PDK_ROOT)/gf180mcuD/libs.tech/magic/gf180mcuD.tech
```

With this in place, simply typing `magic` from any directory will automatically load the `gf180mcuD` technology without requiring the `-rcfile` argument each time.

Command	Purpose
`:drc why`	Explain DRC error inside current box
`:drc find`	Jump box to next DRC error
`:extract all`	Extract netlist from layout
`:ext2spice`	Write SPICE from extraction database
`:ext2sim`	Write .sim file for IRSIM
`:gds write inv`	Export GDS file
`klayout inv.gds`	View or print GDS with KLayout

Common Paint Commands for gf180mcuD

Command	What it draws
`:paint nwell`	N-well (pMOS body)
`:paint ndiff`	N-type diffusion
`:paint pdiff`	P-type diffusion
`:paint poly`	Polysilicon gate
`:paint ndc`	N diffusion contact
`:paint pdc`	P diffusion contact
`:paint nsc`	N substrate contact
`:paint psc`	P well contact
`:paint pc`	Poly contact
`:paint metal1`	Metal 1
`:paint metal2`	Metal 2
`:paint via1`	Via between metal1 and metal2

Additional Resources

- Magic documentation and tutorials: http://opencircuitdesign.com/magic
- gf180mcuD PDK documentation: https://gf180mcu-pdk.readthedocs.io
- open_pdks repository (PDK installer): https://github.com/RTimothyEdwards/open_pdks
- IIC-OSIC-TOOLS Docker container: https://github.com/iic-jku/iic-osic-tools
- Course examples, scripts, and layout guides: https://stineje.github.io
- Video walkthroughs: https://www.youtube.com/user/jlstine/